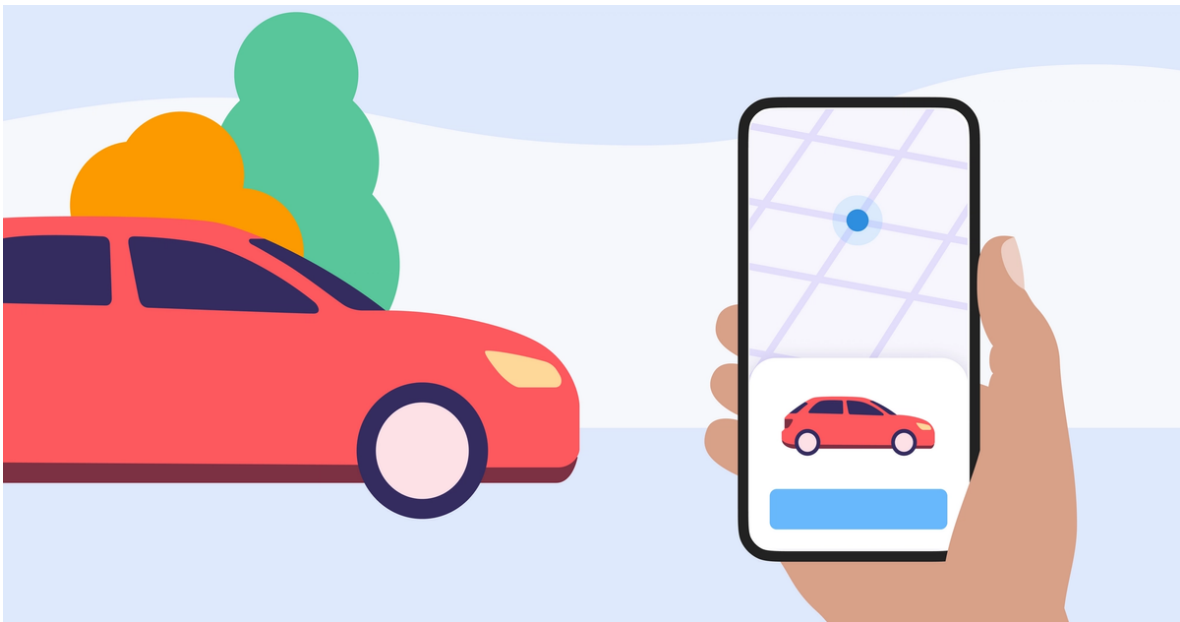




BACHELOR PROJECT

Development of a framework for comparing ride-hailing services in a microscopic traffic simulator



Luca Liuzzi
Fletcher Collis

June 24, 2024

Contents

1	Introduction	2
2	Background	2
2.1	SUMO	2
2.2	Manhattan Grid	3
2.3	Ingolstadt	4
3	Algorithms	5
3.1	Assumptions	5
3.2	First come first served	5
3.3	Distance based algorithm	6
3.4	Mixed distance and FCFS algorithm	6
3.5	Time based algorithm	6
4	Results	9
4.1	Manhattan grid	9
4.2	Ingolstadt	11
5	Conclusion	14
6	Annexe	15

1 Introduction

According to the Oxford english dictionary [1] ride-hailing is : "The use of a mobile app or website to engage a personal driver to collect and transport a fare-paying customer to a chosen destination"

In today's fast-paced world, ride-hailing apps have dramatically transformed urban mobility, offering a convenient and flexible alternative to traditional transportation methods. These platforms, such as Uber and Bolt, leverage advanced algorithms and artificial intelligence to efficiently match riders with drivers, reshaping how people navigate cities (Rayle et al., 2016 [2]). This shift represents a significant evolution in daily commuting, underscoring the growing integration of technology into transportation systems.

Ride-hailing services have had profound impacts on urban transportation, including shifts in vehicle ownership and public transit usage. Ride-hailing dynamically matches supply and demand by allowing travelers to request car rides in real-time from potential suppliers using a smartphone application (Rayle and al., 2016 [2]). Studies indicate that while this relatively new technology has not significantly reduced vehicle ownership, it has influenced some users to abandon their vehicles (Clewlow and Mishra, 2017 [3]). Moreover, the relationship between ride-hailing and public transit is complex, with some evidence suggesting that these services may both complement and compete with traditional public transit, depending on the context and quality of the transit services available (Clewlow and Mishra, 2017 [3]).

This project explores whether we can efficiently use an open source micro-simulator like SUMO (Simulation of Urban MObility [4]) to simulate and solve problems related to ride-hailing algorithms. While most research either uses experiments or custom built simulators to obtain metrics on the performance of various ride-hailing algorithms, we would like to use an open source simulator to obtain results. SUMO has already been used in various research questions [5] but never in such a setting, therefore it is interesting to see if it can be used to simulate ride-hailing services. By diving into the mechanics of matching algorithms, we also aim to design our own solutions and evaluate their effectiveness. This involves a detailed examination of existing algorithms and an analysis of our simulation results to understand their practical implications.

By integrating SUMO in our project, we aim to simulate the dynamics of ride-hailing services and test the efficiency of different matching algorithms in a controlled virtual environment. This approach allows for the examination of various scenarios and the optimization of ride-hailing operations, potentially leading to improvements in real-world applications. The insights gained from this study could contribute to more efficient and sustainable urban transportation systems. This study could also be used as a framework for testing different and new matching algorithms.

2 Background

2.1 SUMO

SUMO stands for "Simulation of Urban MObility". It is an open-source traffic simulation software that allows users to model and simulate traffic scenarios. It does this by using a digital model of a city's road network and simulating vehicle movements in discretised time steps utilising car following models. The software enables the creation of detailed traffic networks. To run a simulation in SUMO, one has to first create two XML-files: a network file and a routing file. The network file describes the topology of the road network, including signalized junctions, lanes reserved for specific traffic (such as buses or bicycles) and other pertinent elements. The route file specifies the route, or path, that every vehicle trip will take in the network, including the start time of the trip. Since specifying the full path over every vehicle-trip can be tedious, it is common to instead generate a demand file consisting of origin-destination pairs and departure times, and then use tools that come with SUMO to generate the route file.

The software also supports a variety of configuration files that allow for precise adjustments to the simulation environment (car following model parameters, speed of vehicles, ...). These configuration files enable a high level of customization, ensuring that the simulation closely mirrors the required conditions.

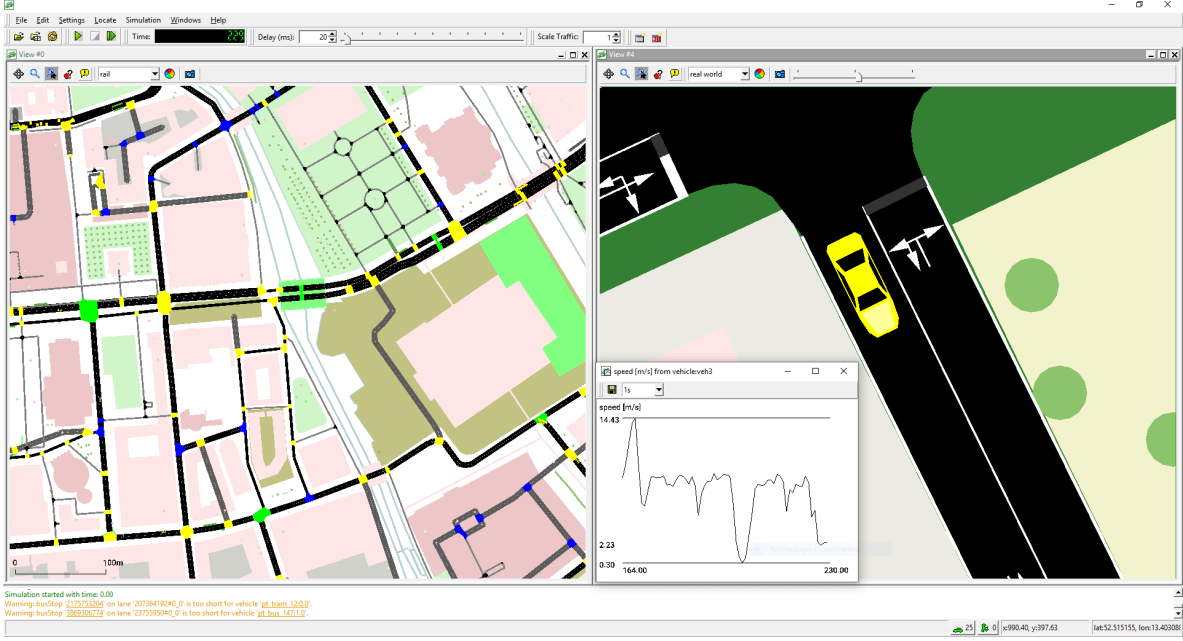


Figure 1: Example of the SUMO graphical interface [4]

Overall, the combination of demand files, route files and configuration files in SUMO provides a decent base for designing and analysing complex traffic scenarios. SUMO can also help with the testing and refinement of ride-hailing algorithms. It can provide valuable insights into their performance, efficiency and impact. This is what we decided to base our project on. An example of what a simulation can look like is shown in Figure 1.

TraCI (Traffic Control Interface [6]) is a library compatible with SUMO. It allows us to interact with the SUMO simulation in real-time, enabling dynamic modifications and detailed control over various elements of the traffic scenario. In our specific case, TraCI enabled us to:

- **Change Vehicle Routes:** Dynamically alter the routes that vehicles take through the network or change their origin/destination based on specific scenarios being tested.
- **Add or Remove Vehicles:** Introduce new vehicles into the simulation or remove existing ones to help simulate a ride hailing service.
- **Retrieve Real-Time Data:** Extract detailed information about vehicle positions, pickup and dropoff times, and other metrics to monitor and analyze the simulation as it runs.

In our project, SUMO and the TraCI library were essential tools that allowed us to simulate real-world situations. We used TraCI to code our matching algorithms, feed them to the simulator and extract metrics. We then used the SUMO software to test them out and make sure they functioned as designed.

2.2 Manhattan Grid

To make sure the algorithms we coded worked as wanted, we first tested them out on a small scale. To do this, we designed a simple Manhattan grid (5x5) which can be seen in Figure 2. A Manhattan grid refers to a street layout characterized by a regular grid pattern of perpendicular streets forming a rectangular network of blocks. This type of street layout is commonly found in urban areas, particularly in cities like New York City, where it originated. By using such a simple network, the load time is much reduced, which allowed us to debug our code more efficiently.

In these simulations we created our own demand list by randomly picking origin/destination pairs (O-D pairs), then assigning a random request time for each of these pairs. We did this using a simple python script that extracted from the network file random edges of our grid to be the O-D pairs. In

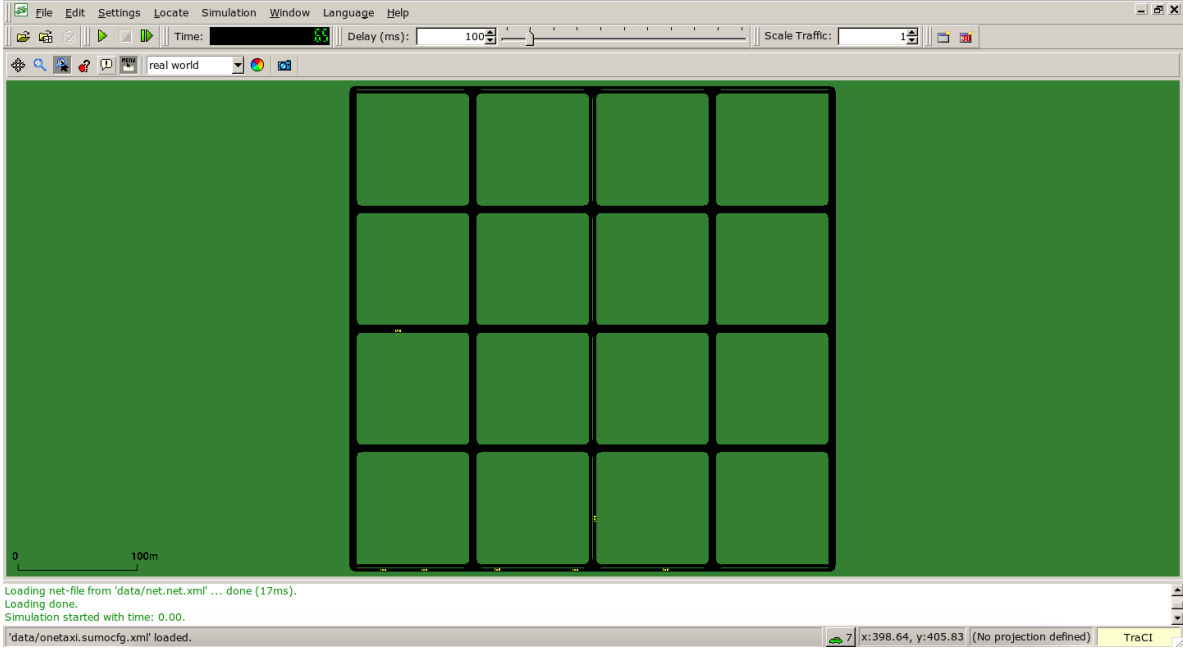


Figure 2: Manhattan grid used

SUMO, an edge is a piece of road that connects two points. It is a segment of the street where cars drive from one spot to another in one direction.

Throughout our experiments we then chose different sample sizes of this O-D list to emulate different fleet size to demand ratios.

2.3 Ingolstadt

Thanks to SUMO's open source capability, we were able to find a real scenario describing the small city of Ingolstadt [7] located in Bavaria, Germany. In Figure 3 we can see an image of the city and its location. The scenario contains a detailed description of the city's road network, a configuration file describing the simulation's settings and a 24h vehicle demand, including which path each single vehicle will take. The scenario has been calibrated based on real traffic measurements..

In this simulation, we extracted a random sample of OD pairs and request times from the huge 24h long route file, once again using a simple python script.



Figure 3: The city centre of Ingolstadt

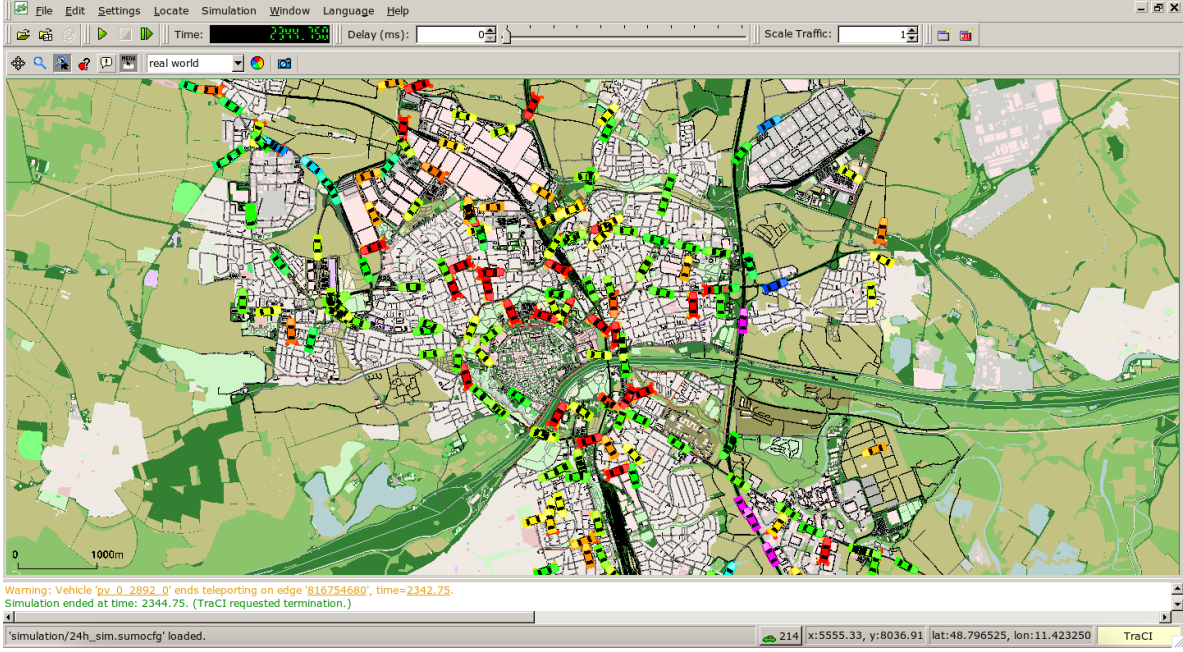


Figure 4: Ingolstadt network in SUMO

3 Algorithms

3.1 Assumptions

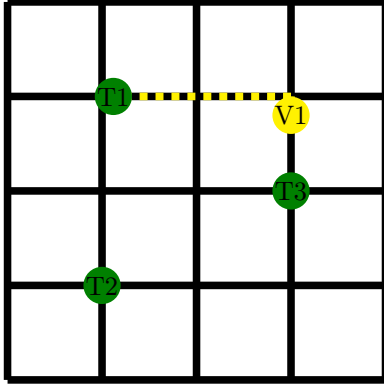
To design our different algorithms, we had to make certain assumptions. First of all, we assume that the starting point for all ride-hailing vehicles is the same and set by us. For the Ingolstadt scenario we chose a big parking close to the center, as this seemed to be a realistic possibility. For the Manhattan grid we chose a random edge. Then, we define the pick up point to be the start of an edge in the network and the drop off point to be the end of an edge. We also assume that if a vehicle is not assigned anymore requests, it parks at its last known location. Finally, travel time is calculated by using the "findRoute" function in the TraCI library. This function finds the fastest route between an OD pair with current traffic conditions and returns the travel time. These assumptions are made to make it easier to model the various algorithms in the following sections.

3.2 First come first served

In the realm of ride-hailing services, the First-Come, First-Served (FCFS) algorithm is probably the simplest method for matching passengers with available drivers.

As its name suggests, FCFS prioritizes requests based on the time they are made, with the earliest request receiving precedence. When a passenger is dropped off by a taxi, the next passenger to be picked up is the one who submitted the request the earliest and who has not yet been assigned a driver.

This simplistic approach aims to minimize passenger wait times by ensuring that the first request in the queue is promptly addressed. However, even though this may be its aim, often it is not very efficient as it does not take into account the distance between where the request is made and the assigned driver. Nonetheless, FCFS serves as a baseline algorithm for evaluating more complex matching strategies and remains a valuable benchmark in ride-hailing service optimization.



Vehicle empty

New request

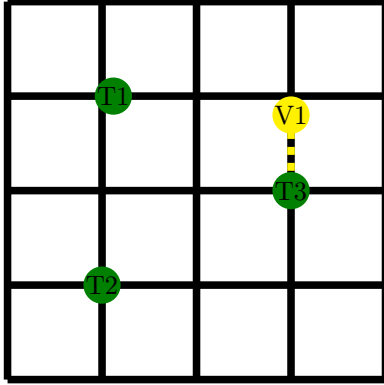
$T1 < T2 < T3$ are request times

Once a vehicle (V1) is empty, it goes to pick up the request with the earliest request time (T1)

3.3 Distance based algorithm

To try and find a more efficient ride-hailing algorithm, we then tried to take distance into account since this seemed to be the main problem with FCFS.

Therefore, in contrary to FCFS, here when a passenger is dropped off by a taxi, the next passenger to be picked up is the one who is closest to the drop off location.



Vehicle empty

New request

$T1 < T2 < T3$ are request times

Once a vehicle (V1) is empty, it goes in search of the closest passenger in terms of travel time without respecting the time order of requests (T3)

3.4 Mixed distance and FCFS algorithm

When running the Distance-based algorithm, we noticed that from time to time some requests took a lot of time to be completed as their origin was quite far away from the other trips. We therefore mixed up FCFS and the distance algorithm here. The main algorithm is the distance algorithm, as described previously. However if a request is not taken after a fixed amount of time, $T_{\max}$, we switch to a FCFS for that particular request. If there are multiple requests for which the waiting time is larger than $T_{\max}$, they are simply treated one after another in the same way as the FCFS algorithm until all current requests have a waiting time lower than $T_{\max}$.

3.5 Time based algorithm

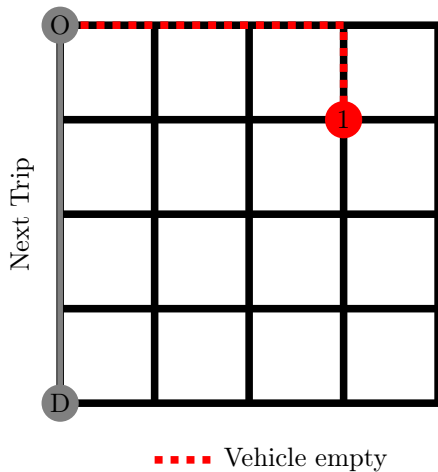
As an alternative to the other three algorithms, we developed a fourth algorithm based on the moment the request is made. The major difference of this option from the other three is that the trip is

assigned to a vehicle at the time the request is made instead of being assigned based on the availability of ride-hailing vehicles.

When a trip request is made, the time it takes each vehicle in the fleet to reach the pick-up point is determined. The trip will be assigned to the vehicle that takes the least amount of time to reach the pick up location. To determine the time it takes to reach the pick-up point, the time to finish the current route and any other trips assigned to the vehicle must be taken into account. The travel time between two points is calculated ignoring potential congestion. The calculation differs slightly depending on the state of the vehicle (parked, traveling with passenger, traveling empty). We explain below, with the help of the some schemas, how the travel time to the pick up location is calculated depending on the different states of the vehicle.

Parked vehicle:

This is the simplest case, in fact the vehicle has no trip scheduled and therefore is parked at the drop off location of its last trip. When a request arrives, the free flow travel time between the parked location and the origin of the new trip is calculated.

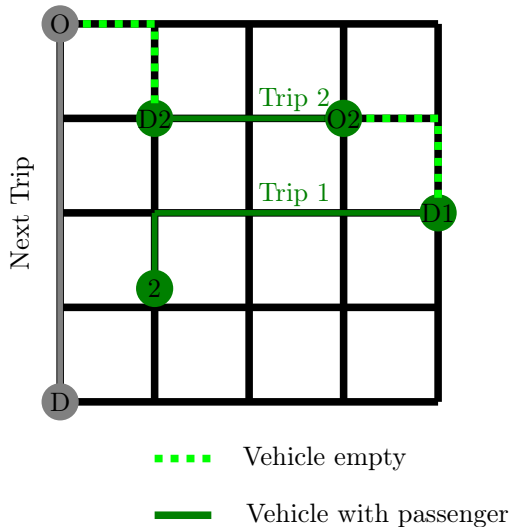


The vehicle is parked at point 1. The new trip starts at the Point O and ends at Point D.

Compute travel time:
from Point 1 to Point O

Vehicle travelling with passenger:

In this case the vehicle has a passenger and is on its way to its current passenger's destination. The first step is to determine the travel time between the vehicle's position at the time of the request and the destination of the current route. Next, one has to take into account the fact that the vehicle may have other trips assigned, so one has to calculate the travel time in free flow conditions to the origin of the next trips, followed by their duration.



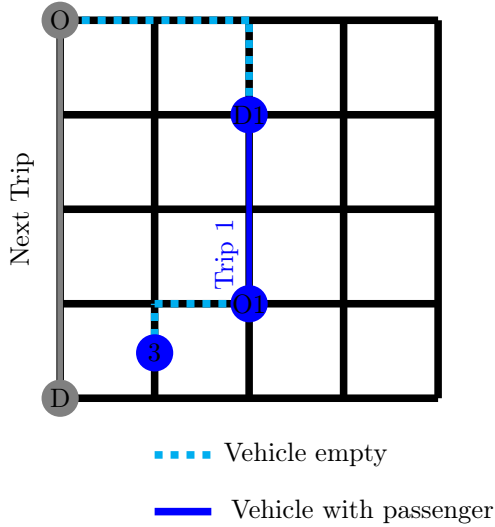
The vehicle is at point 2 and is in the direction of point D1. In addition to "trip1" it has been assigned "trip2". The new trip starts at the Point O and ends at Point D.

Compute travel time:

- (i) from Point 2 to Point D1
- (ii) from Point D1 to Point O2
- (iii) from Point O2 to Point D2
- (iv) from Point D2 to Point O

Vehicle on an empty journey:

The last possible case occurs when the vehicle already has requests assigned, and is en route to the next pick-up point but does not currently have a passenger. Before it can proceed to the origin of the newly made request, it must complete its list of outstanding trips. This situation is very similar to the previous one, with the difference being that the first travel time to be calculated is between the current position of the vehicle and the origin of its next trip, which is already in the outstanding trip list.



The vehicle is at point 3 and is in the direction of point O1 to take the "trip1". The new trip starts at the Point O and ends at Point D.

Compute travel time:

- (i) from Point 3 to Point O1
- (ii) from Point O1 to Point D1
- (iii) from Point D2 to Point O

4 Results

4.1 Manhattan grid

To help analyse the outputs of our various simulations and the performance of each algorithm, we computed the mean waiting time for a demand of 100 requests, with varying fleet sizes represented as percentages of the demand size. The results are shown in the following tables. The simulations were run over two different time periods: 1000 seconds in Table 1 and 1728 seconds in Table 2. The time period in Table 2 was chosen to easily be able to draw parallels with our Ingolstadt simulation results as we wanted the request - time ratio to be similar as to not skew our comparisons.

Fleet size	5	10	15	20	25	30
FCFS	835.5	213.2	71.95	64	64	64
Distance based	835.6	222.2	67.35	67.24	67.24	67.24
Mixed (100 second max wait)	847.67	221.24	67.35	67.24	67.24	67.24
Request based	811.67	208.91	75.14	61.24	60.84	60.84

Table 1: Mean waiting time [seconds] for 100 requests over 1000 seconds with varying fleet sizes

Fleet size	5	10	15	20	25	30
FCFS	533.62	86.15	62.13	61.98	61.98	61.98
Distance based	559.2	85.93	61.06	61.98	61.98	61.98
Mixed (200 second max wait)	544.17	85.93	61.06	61.98	61.98	61.98
Request based	548.86	72.06	48.11	47.31	47.31	47.31

Table 2: Mean waiting time [seconds] for 100 requests over $864 \times 2 = 1728$ seconds with varying fleet sizes

From the results in Table 1 and Table 2, we can observe some interesting traits. First of all, as the fleet size grows, the wait time generally decreases. This happens because more vehicles become available to pick up demands. As a result, the time waited for a vehicle to finish its current demand before taking on a new one becomes smaller. Instead, an empty vehicle is available to take the next demand. We can also see that most strategies stabilize in terms of mean waiting time at fleet sizes of 20% of the total number of requests or higher. Stabilization indicates that the fleet is large enough so that it does not operate at full capacity, and hence other factors than the time it takes for a vehicle to become empty, such as the travel time for the passenger, become more significant.

When looking at the results in more detail, we can also see that at the smallest fleet size of 5%, all strategies exhibit very high mean waiting times. It seems that when the demand greatly outweighs the supply, the strategy does not have much of an effect. As the Manhattan grid used is quite small, this could also play a role in making the performances similar for each algorithm at a low supply/demand ratio.

Once we begin to increase the fleet size of our supply, we can start to see that some algorithms are more efficient than others. As expected, the FCFS strategy is generally the worst in terms of waiting times for passengers. This is due to its simplistic approach. We can then notice that the Mixed and Distance based algorithms perform differently with low fleet sizes and then perform the same as the fleet size increases. This occurs because the max wait limit imposed actually comes in to effect. At low supply/demand ratios, the limit is met, resulting in a switch back to FCFS for the passenger in question. However, once the fleet size grows, the limit time is never met meaning it has no algorithmic difference from the Distance based algorithm. The results also show that, when looking at the mean waiting time for all passengers, the Mixed strategy seems better. It manages to avoid huge one-off waiting times, which is a good thing.

Overall, the request based algorithm performs the best. Whether with small or large supply fleet sizes it generally out performs all other algorithms in terms of mean waiting time. This strategy is the most complex and the most well thought out, this comes as no surprise.

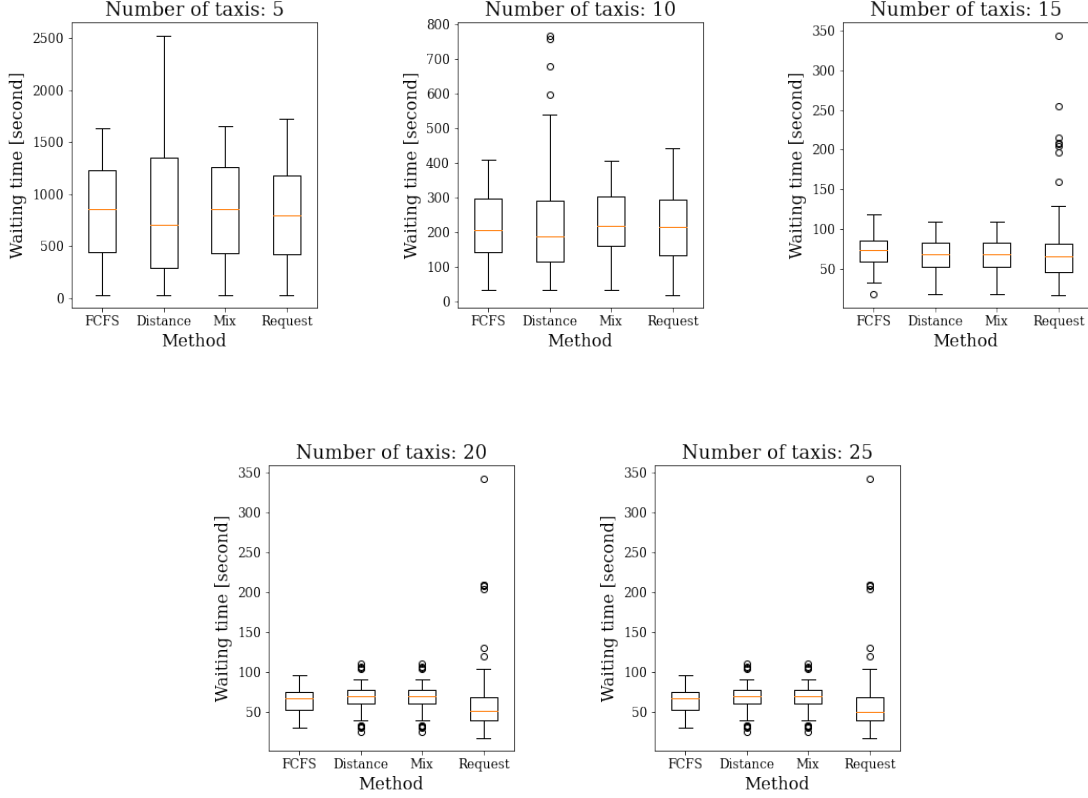


Figure 5: Boxplots of the distribution of waiting time (demand = 100 / 1000 seconds)

Figure 5 shows the box plots of our different scenarios, giving a more in-depth view on the performance of our models. Similar traits as before are visible, however now we are able to visualise the standard deviation and distribution of the waiting time among passengers.

As the fleet size increases, the box size and whisker lengths are reduced. This is the case for all algorithms and indicates that for the majority of passengers, the waiting time is more and more similar and tends to become smaller. At small fleet sizes however, the whiskers of the box plots are quite long, indicating that the waiting time is not equally distributed between all the users. We can see this particularly with the Distance based algorithm. Some passengers who's trips start at more remote locations are simply forgotten about and end up waiting for much longer periods of time than the average. At these small fleet sizes, all algorithms perform similarly badly.

It is also worth noting that the plots for 20% and 25% supply fleet sizes are exactly the same. This highlights the so called stabilisation point mentioned previously and shows us that past this point, augmenting the fleet size will have no effect on the waiting time experienced by passengers.

As before, the difference between the Distance based algorithm and the Mixed algorithm can no longer be noticed from 15% fleet size and onwards. However, the box plots do reveal that the Request based algorithm may not perform as well as first thought for everyone. The box plot of this method shows quite a few outliers who wait for large amounts of time. It also shows that even though the mean waiting time is low, the standard deviation is higher than with other algorithms, indicating an unequal treatment of some passengers.

To conclude, these insights emphasize the importance of considering both mean waiting times and variability when evaluating the performance of ride-hailing algorithms. Ensuring a balance between efficiency and consistency is crucial for maintaining high service quality and passenger satisfaction.

4.2 Ingolstadt

To complete our analysis and provide a more realistic scenario than the previously shown Manhattan grid, we applied the same methodology as before to the Ingolstadt simulation over a 24h time period. Although we encountered some issues, they were not too serious and were to be expected as the open source Ingolstadt scenario was not conceived for such use.

First, certain OD (origin-destination) pairs from the demand file caused the simulation to crash due to dead-end destinations. Vehicles would get stuck when attempting to travel to the origin of their next assigned trip. To fix this, we updated our Python scripts to teleport stuck vehicles to the nearest valid road, allowing the simulation to continue while still remaining realistic.

Additionally, when running the simulation with traffic, some trips were not completed. This was likely due to SUMO’s teleportation feature, which moves stuck vehicles to keep the simulation running. This can result in trips being marked as incomplete, especially in heavy traffic.

To address this, we modified our analysis to consider only the trips that were actually executed. This allowed us to account for ‘lost’ trips that were planned but not completed due to traffic and teleportation issues. This fix allowed us to obtain some results (c.f. Annexe) but not enough to fully analyse the different algorithms when traffic is present. We therefore also decided to run the simulation without traffic. By doing this, we were able to obtain better results and these are the values we used in the following analysis (Tables 3 and 4).

Fleet size	50	75	100	125	250	500
FCFS	24530	7461	1190	586	582	590
Distance based	24951	7643	1214	586	590	590
Mixed (1500 second max wait)	24531	7690	1175	589	590	590
Request based	24064	7330	1337	709	603	477

Table 3: Mean waiting time [seconds] for a demand of 5000 and varying fleet size with no traffic

	50	75	100	125	250	500
FCFS	5000	5000	5000	5000	5000	5000
Distance based	5000	5000	5000	5000	5000	5000
Mixed (1500 second max wait)	5000	5000	5000	5000	5000	5000
Request based	4979	4979	4979	4979	4979	4979

Table 4: Effective demand with no traffic

From the results in Table 3, we can observe some interesting phenomena. The main behavioral traits are once again evident.

As the fleet size increases, the waiting time for users decreases. We can also observe something similar to the stabilization point seen in the Manhattan grid scenario. Here we can see a stagnation in the mean waiting time when the fleet size is at around 250 for the first three algorithms. This indicates that an increase in the fleet size has no effect on overall performance. The Request based algorithm also experiences this, but with a higher fleet size. Its stabilisation point is not visible in table 3, but is in fact around the 500 fleet size. This is the same phenomena described previously.

When diving deeper into these results, we can again see some familiar traits. At the smallest fleet size, all strategies exhibit very high mean waiting times reinforcing the idea that the strategy does not have an effect when the demand heavily outweighs the supply. This therefore also seems to be true when the size of the network is large. However, as soon as we increase the fleet size, these values reduce significantly and we can start to see the difference in the performance of the various algorithms.

Surprisingly, the FCFS algorithm actually has one of the best performances in terms of mean waiting time at most fleet sizes. This is in opposition to the results found when using the Manhattan grid and could be explained by the different nature of the road network in Ingolstadt. The more complex

and less grid-like structure of the Ingolstadt network may favor simpler algorithms like FCFS, which prioritize the first available request rather than optimizing for distance or request specifics.

Additionally, the Request-based algorithm, while first thought to be an effective strategy and having the best results at higher fleet sizes (fleet ≤ 75), seems to show higher waiting times once the fleet size decreases, then, at a fleet size of 500 it once again has the best result. This bad performance with a medium sized fleet might be due to the specific routing challenges and traffic patterns encountered in the Ingolstadt scenario. We can also see in Table 4 that there are 21 missing trips for the request base strategy. This is due to the algorithm itself which cannot seem to process multiple requests with the same request time, resulting in these trips being simply forgotten. This means that in reality, the waiting times should be slightly higher, as there is more demand, but with such large demand, this is negligible.

The Mixed algorithm, which incorporates both distance and wait time considerations, results in consistent performance. It closely matches the Distance-based algorithm, but its max wait step limit seems to give it an upper hand. This consistency across different scenarios indicates robustness, making it a reliable choice even when the network conditions vary.

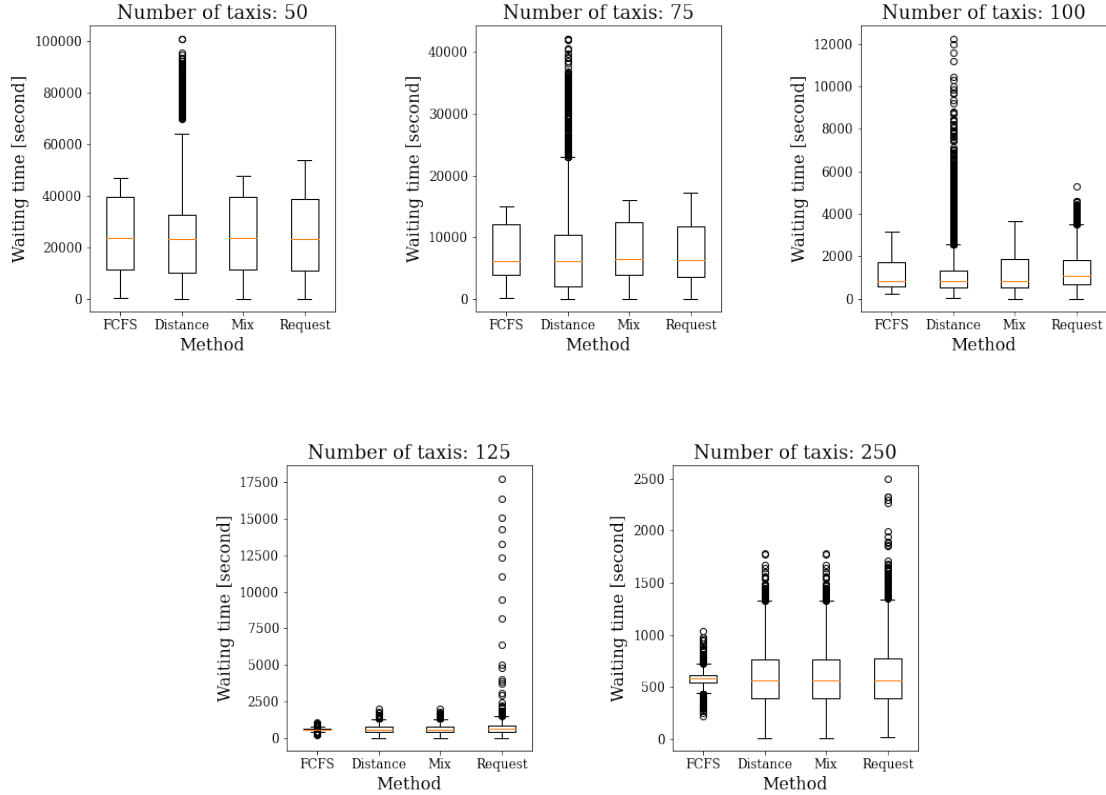


Figure 6: Boxplots of the distribution of waiting time with no traffic (demand = 5000 / 86400 seconds)

In Figure 6 we can see the box plots of the waiting times of our different scenarios and algorithms.

Again, the box plots illustrate that a larger fleet size initially significantly reduces waiting times. This effect diminishes as the fleet size increases, reaching a stabilization point at a fleet size of around 250 for the FCFS, Distance based and mixed algorithms.

The FCFS strategy is particularly efficient at all fleet sizes. The short whiskers and minimal outliers in its box plots indicate that most passengers experience wait times very close to the average, with minimal variation. This suggests that the FCFS strategy is effective in ensuring fairness among users in this scenario.

In contrary, we can see that the Request based algorithm seems to perform badly compared to the FCFS. While it boasts a lower average wait time for users at small fleet sizes (≤ 75), the high standard

deviation in its box plots reveals a significant drawback: inequality. This suggests that while some users might experience a much faster and more responsive service, others might face significantly longer wait times. The box plots also reveal that there are many outliers when the fleet size increases. This contributes to the unsatisfactory mean waiting times and this trend seems to become particularly concerning as the fleet size increases. While it prioritizes specific requests and offers a potential efficiency gain in some scenarios, the Request-based approach seems to suffer from a lack of stability and fairness when dealing with larger workloads.

When looking at the Distance based and Mixed algorithms, we can see the same trends as in the Manhattan grid scenario. At small fleet sizes, the Mixed algorithm has the upper hand. It manages to avoid outliers and big waiting times thanks to the max wait time. The Distance based algorithm on the other hand, has many users who wait excessively. From a fleet sizes of 125 and bigger, both are exactly the same highlighting again, the effect of the max waiting time.

In conclusion, we can say that a simple strategy like FCFS can actually perform reasonably well depending on the scenario. However, the nuances of each algorithm's performance at different fleet sizes reveal areas for further exploration and optimization. While a deeper analysis of our algorithms could yield additional insights, we have successfully achieved our primary goal of being able to simulate and compare their effectiveness in various scenarios.

5 Conclusion

To conclude, our research into the use of SUMO, an open-source program, as a micro simulation tool for ride-hailing services has produced encouraging findings. We were able to assess the performance of different ride-hailing algorithms under desired conditions and simulate them efficiently using this tool.

Research into the various ride-hailing algorithms we coded has confirmed a few points worth noting. As the fleet size grows, the waiting time experienced by users decreases. This highlights the importance of fleet size optimisation in ride-hailing applications. The stabilization point observed indicates that beyond a certain size of the fleet, an increase in the supply barely reduces waiting time, thereby underlining the importance of strategic management of the fleet size as opposed to concentrating merely on the growth of the fleet.

Our analysis also reveals that while simplistic approaches like FCFS may seem less effective, in reality their performance can be surprising. More sophisticated algorithms can also significantly enhance efficiency, particularly when the fleet size is not too small. This underscores the potential of advanced matching algorithms and their role in optimizing how ride-hailing services function.

We have showed that open-source simulators can be powerful resources in transportation research by incorporating SUMO into our project. These simulators offer an affordable and adaptable platform for testing and creating new algorithms, while remaining open source, meaning that they are largely accessible. This strategy not only provides useful advantages for urban transportation, but we hope it also establishes the framework for further studies in this area.

Subsequent research may build upon this framework by integrating empirical data, investigating supplementary factors, and evaluating novel algorithms to enhance and maximise ride-hailing's performance. We hope to have helped contribute to smarter, more effective transport systems and better meet the changing demands of urban mobility by proving that this easily accessible software can be used.

6 Annexe

Ingolstadt with traffic : Results

Fleet size	50	75	100	125	250
FCFS	30201	12287	3687	1147	683
Distance based	-	12540	4020	1332	690
Mixed (1500 second max wait)	-	12568	4081	1267	690
Request based	3629	11755	2912	890	790

Table 5: Mean waiting time for 5000 requests over 24 hours with varying fleet sizes

	50	75	100	125	250
FCFS	5000	4999	5000	4999	4998
Distance based	-	4999	4998	4995	4998
Mixed (1500 second max wait)	-	4999	4998	4998	4998
Request based	870	4979	3809	3050	3214

Table 6: Effective demand

References

- [1] Oxford English Dictionary. *Ride-Hailing*. URL: https://www.oed.com/dictionary/ride-hailing_n?tab=meaning_and_use#1330728810100.
- [2] Lisa Rayle et al. “Just a Better Taxi? A Survey-Based Comparison of Taxis, Transit, and Ridesourcing Services in San Francisco”. In: *Transport Policy* (2016).
- [3] Regina R. Clewlow and Gouri Shankar Mishra. *Disruptive Transportation: The Adoption, Utilization, and Impacts of Ride-Hailing in the United States*. Tech. rep. Institute of Transportation Studies, University of California, Davis, 2017.
- [4] Eclipse SUMO. *SUMO - Simulation of Urban MObility*. URL: <https://github.com/eclipse-sumo/sumo>.
- [5] DLR Institute of Transportation Systems. *SUMO Documentation and Publications*. <https://sumo.dlr.de/docs/Publications.html>.
- [6] DLR (German Aerospace Center). *TraCI (Traffic Control Interface) Documentation*. <https://sumo.dlr.de/docs/TraCI.html>.
- [7] AUDI AG, Chair of Traffic Engineering, and Control of the Technical University of Munich. *Multimodal Traffic Simulation of Ingolstadt in SUMO*. <https://github.com/TUM-VT/sumo-ingolstadt>.